PlaceMatch: Campus Placement Management with Semantic Resume–JD Matching

DBMS Experiential Learning, Level 3 Project Report Department of Computer Science and Engineering, PES University

Team member	SRN
Swaroop Venkateshwar	PES1UG24CS923
Shreesh S A	PES1UG24CS441

Repository: https://github.com/swaroop05v/placematch

1. Objectives

Campus placements are usually coordinated through spreadsheets, forms and mailing lists. Eligibility is checked by hand, students apply to drives that don't fit them, recruiters skim hundreds of resumes, and nobody has a live view of placement statistics. PlaceMatch replaces that with a database-centred web application that:

1. **Models the full placement workflow**: companies, recruiters, drives, eligibility rules, interview rounds, results and offers, with the rules enforced inside the database.
2. **Enforces placement policy automatically**: CGPA, backlog, batch and department criteria, plus the "one student, one job (Dream exception)" rule.
3. **Matches people to jobs by meaning, not keywords**: resumes and job descriptions are embedded into vectors so the system can recommend drives, rank applicants and search talent semantically.
4. **Gives each role its own interface**: students, recruiters and the placement office (admin), each with only the permissions it needs.
5. **Keeps data consistent and traceable** through transactions, triggers and an audit log.

2. Functional requirements

#	Role	Requirement
F1	All	Log in with email and password, with a role-based home page. Unauthenticated users are redirected to the login page.
F2	Student	Self-register with SRN, department, batch, CGPA and backlogs (validated by DB constraints).
F3	Student	Maintain resume text, contact details and skills. Saving recomputes the resume embedding.
F4	Student	Browse open drives with keyword (full-text) or semantic search and filter by tier and job type. See per-drive eligibility with the reason.
F5	Student	See recommended drives ranked by resume–JD similarity, restricted to drives they are eligible for.
F6	Student	Apply to and withdraw from drives, and track round-by-round progress.
F7	Student	Accept or decline offers. Accepting applies the placement policy automatically.
F8	Recruiter	Create drives (JD, CTC, eligibility, departments, skills, ordered rounds) as draft or published. Publish, close and complete drives.

#	Role	Requirement
F9	Recruiter	View applicants ranked by semantic match. Record round results; the final round creates an offer automatically.
F10	Recruiter	See eligible students who haven't applied but match the JD. Run a semantic talent search alongside a keyword search.
F11	Admin	Dashboard with department-wise placement %, company offer stats, tier distribution, and students needing support.
F12	Admin	Verify and correct student CGPA and backlogs. Manage companies and recruiter accounts. Manage the status of any drive.
F13	Admin	View the audit log of all changes to applications, offers and drives.
F14	Admin	SQL Lab: run the project's important queries live in a read-only transaction.

3. Database choice and justification

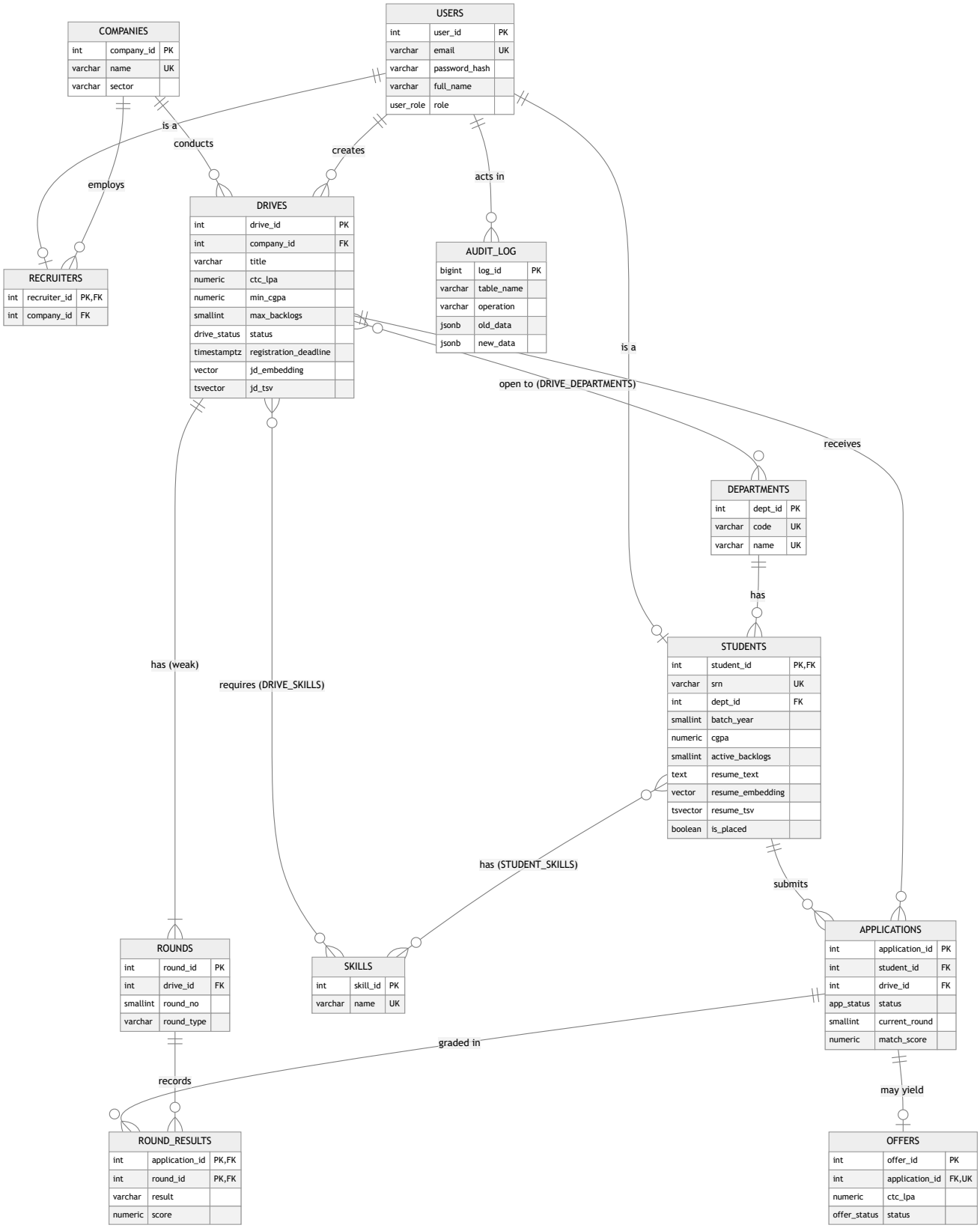
PlaceMatch uses **Option 2: SQL + vector database**, with **PostgreSQL** for the relational data and the **pgvector** extension, on the same Neon PostgreSQL instance, for embeddings.

Kind of data	Store	Why
Users, students, companies, drives, rounds, applications, results, offers	**PostgreSQL tables**	Highly structured, with many relationships (1:1, 1:N, M:N) and strict business rules. Accepting an offer must update several tables atomically. These needs (normalisation, foreign keys, CHECK constraints, ACID transactions, triggers, joins and aggregates for reporting) are exactly what a relational DBMS is for. A document store would push those rules into application code.
Resume text and job-description text	**PostgreSQL `text` + `tsvector`**	Needed for display and for exact keyword search (full-text search with GIN index).
Meaning of resumes and job descriptions	**pgvector `vector(384)` + HNSW index**	Unstructured text can't be compared with `=` or `LIKE`. "Built a RAG chatbot with sentence embeddings" and "LLM / NLP engineer" share almost no keywords but mean similar things. Embeddings put semantically similar texts close together, so *k*-nearest-neighbour search by cosine distance answers "who fits this job?" and "which jobs fit me?".

Why pgvector rather than a separate vector database (Qdrant, Pinecone, and so on)? The vectors belong to rows that already live in PostgreSQL (one per student, one per drive), and every vector query is combined with relational filters such as eligibility, open status and department. Keeping both in one engine means:

- a single query can join, filter and rank by similarity, for example `WHERE fn_check_eligibility(...) ORDER BY embedding <=> ...`;
- the embedding is updated in the **same transaction** as the resume text, so they can never drift apart. A trigger also clears a stale embedding;
- there is one connection, one backup and one security boundary.

4. ER diagram



Key design decisions

- **Specialisation (ISA)**: `students` and `recruiters` are 1:1 sub-types of `users` and share its primary key (`student_id = user_id`), so authentication lives in one table while role-specific attributes stay in their own tables.

- **Weak entity**: a `round` exists only within a drive, identified by `(drive_id, round_no)` (UNIQUE), with ON DELETE CASCADE.
- **M:N relationships** (`student_skills`, `drive_skills`, `drive_departments`) are resolved with junction tables using composite primary keys.
- **Associative entity**: `applications` connects students and drives (UNIQUE `(student_id, drive_id)`) and carries its own state. `round_results` connects applications and rounds.
- **1:0..1**: an application has at most one `offer` (UNIQUE `application_id`).

5. Relational schema

Primary keys are **bold**. Foreign keys are marked with †.

- USERS(**user_id**, email, password_hash, full_name, role, is_active, created_at)
- DEPARTMENTS(**dept_id**, code, name)
- STUDENTS(**student_id**†→USERS, srn, dept_id†→DEPARTMENTS, batch_year, cgpa, active_backlogs, phone, linkedin_url, resume_text, resume_embedding, resume_tsv, is_placed, updated_at)
- SKILLS(**skill_id**, name)
- STUDENT_SKILLS(**student_id**†→STUDENTS, **skill_id**†→SKILLS, proficiency)
- COMPANIES(**company_id**, name, sector, website, about)
- RECRUITERS(**recruiter_id**†→USERS, company_id†→COMPANIES, designation)
- DRIVES(**drive_id**, company_id†→COMPANIES, created_by†→USERS, title, job_type, description, location, ctc_lpa, min_cgpa, max_backlogs, batch_year, status, registration_deadline, jd_embedding, jd_tsv, created_at)
- DRIVE_DEPARTMENTS(**drive_id**†→DRIVES, **dept_id**†→DEPARTMENTS)
- DRIVE_SKILLS(**drive_id**†→DRIVES, **skill_id**†→SKILLS)
- ROUNDS(**round_id**, drive_id†→DRIVES, round_no, name, round_type, scheduled_at); UNIQUE(drive_id, round_no)
- APPLICATIONS(**application_id**, student_id†→STUDENTS, drive_id†→DRIVES, status, current_round, match_score, applied_at, updated_at); UNIQUE(student_id, drive_id)
- ROUND_RESULTS(**application_id**†→APPLICATIONS, **round_id**†→ROUNDS, result, score, remarks, evaluated_by†→USERS, evaluated_at)
- OFFERS(**offer_id**, application_id†→APPLICATIONS (UNIQUE), ctc_lpa, status, offered_at, respond_by, responded_at)
- AUDIT_LOG(**log_id**, table_name, operation, row_pk, actor_id, old_data, new_data, logged_at)

Normalisation. Every table is in 3NF/BCNF. Each non-key attribute depends on the whole key and nothing but the key. Multi-valued facts (skills, eligible departments) live in separate tables rather than comma lists. Derived values are not stored redundantly: tier comes from `fn_drive_tier(ctc)`, and counts come from views. The two exceptions are deliberate. `match_score` is a snapshot of the similarity at application time, and `is_placed` is a flag maintained by the accept-offer procedure for fast filtering.

6. Data dictionary

users

Column	Type	Constraints	Description
user_id	SERIAL	PK	Surrogate key

Column	Type	Constraints	Description
email	VARCHAR(120)	NOT NULL, UNIQUE, CHECK format, CHECK lowercase	Login id
password_hash	VARCHAR(100)	NOT NULL	bcrypt hash (never the plain password)
full_name	VARCHAR(80)	NOT NULL, CHECK length ≥ 2	
role	user_role ENUM	NOT NULL	admin / recruiter / student
is_active	BOOLEAN	DEFAULT TRUE	Disabled accounts can't log in
created_at	TIMESTAMPTZ	DEFAULT now()	

students

Column	Type	Constraints	Description
student_id	INT	PK, FK → users ON DELETE CASCADE	Same id as the user
srn	VARCHAR(13)	UNIQUE, CHECK `^PES[12]UG[0-9]{2}[A-Z]{2}[0-9]{3}$`	University SRN
dept_id	INT	FK → departments ON DELETE RESTRICT	
batch_year	SMALLINT	CHECK 2020–2035	Graduating year
cgpa	NUMERIC(4,2)	CHECK 0–10	Verified by admin
active_backlogs	SMALLINT	CHECK ≥ 0	
phone	VARCHAR(10)	CHECK Indian mobile format	Optional
linkedin_url	VARCHAR(150)	CHECK linkedin.com URL	Added via ALTER TABLE
resume_text	TEXT	NOT NULL	Free text used for search
resume_embedding	vector(384)	HNSW index	Embedding of resume + skills
resume_tsv	tsvector	GENERATED ALWAYS … STORED, GIN index	Full-text vector
is_placed	BOOLEAN	DEFAULT FALSE	Set by `sp_accept_offer`

drives

Column	Type	Constraints	Description
drive_id	SERIAL	PK	
company_id	INT	FK → companies ON DELETE CASCADE	
created_by	INT	FK → users ON DELETE SET NULL	Recruiter who created it
title, description, location	VARCHAR / TEXT	NOT NULL	
job_type	VARCHAR(12)	CHECK IN ('FTE','Internship','Intern+FTE')	
ctc_lpa	NUMERIC(6,2)	CHECK 0 < ctc < 500	Tier: Dream ≥ 20, Core ≥ 10, else Mass
min_cgpa / max_backlogs	NUMERIC / SMALLINT	CHECK ranges	Eligibility criteria

Column	Type	Constraints	Description
batch_year	SMALLINT		Eligible graduating batch
status	drive_status ENUM	DEFAULT 'draft'	draft → open → closed → completed
registration_deadline	TIMESTAMPTZ	NOT NULL	
jd_embedding	vector(384)	HNSW index	Embedding of title + JD + skills
jd_tsv	tsvector	GENERATED (title weight A, description B), GIN	

applications

Column	Type	Constraints	Description
application_id	SERIAL	PK	
student_id	INT	FK → students (added by ALTER)	
drive_id	INT	FK → drives (added by ALTER)	
—		UNIQUE (student_id, drive_id)	One application per drive
status	app_status ENUM		applied / in_process / selected / rejected / withdrawn
current_round	SMALLINT		Round the candidate is waiting on
match_score	NUMERIC(5,4)	CHECK −1…1	Cosine similarity, stamped by trigger

Other tables

Table	Purpose and key constraints
departments	`code` UNIQUE with CHECK `^[A-Z]{2,6}$`; `name` UNIQUE
skills	`name` UNIQUE
student_skills	PK (student_id, skill_id); proficiency CHECK 1–5
companies	`name` UNIQUE; website CHECK `^https?://`
recruiters	PK = FK → users; company FK ON DELETE **RESTRICT** (a company with recruiters can't be deleted)
drive_departments / drive_skills	Composite PKs, CASCADE on drive delete. No rows in drive_departments means all departments are eligible.
rounds	UNIQUE (drive_id, round_no); round_type CHECK IN ('OA','GD','Technical','HR')
round_results	PK (application_id, round_id); result CHECK IN ('cleared','rejected'); score CHECK 0–100
offers	`application_id` UNIQUE; CHECK respond_by > offered_at; CHECK pending ⇔ responded_at IS NULL
audit_log	JSONB before/after images, actor id, timestamp

7. SQL implementation

All DDL is in `db/` and is run in order by `npm run db:setup`.

7.1 DDL (CREATE / ALTER / DROP)

- `DROP VIEW / TABLE / TYPE IF EXISTS … CASCADE` makes the script re-runnable.
- `CREATE TYPE … AS ENUM` for roles and statuses (domain integrity).
- `CREATE TABLE` with inline PK, UNIQUE, CHECK, DEFAULT, generated columns and FKs with chosen referential actions (CASCADE / RESTRICT / SET NULL).
- `ALTER TABLE … ADD CONSTRAINT … FOREIGN KEY` for the students and applications FKs, and `ALTER TABLE students ADD COLUMN linkedin_url` plus a new CHECK.

7.2 Indexes (`02_indexes.sql`)

Index	Type	Used by
idx_applications_drive (drive_id, status)	B-tree composite	Drive funnel, applicant list
idx_applications_student (student_id, status)	B-tree composite	Student dashboard
idx_drives_open_deadline WHERE status = 'open'	**Partial** B-tree	Browse open drives
idx_students_batch_cgpa (batch_year, cgpa DESC)	B-tree composite	Eligibility filtering
idx_drives_jd_tsv, idx_students_resume_tsv	**GIN**	Full-text search
idx_students_resume_vec, idx_drives_jd_vec	**HNSW** (vector_cosine_ops)	k-NN similarity search

7.3 Functions and stored procedures (`03_functions_triggers.sql`)

Name	Kind	What it does
`fn_drive_tier(ctc)`	SQL function (IMMUTABLE)	Dream / Core / Mass classification
`fn_match_score(student, drive)`	SQL function	`1 - (resume_embedding <=> jd_embedding)`
`fn_check_eligibility(student, drive)`	PL/pgSQL set-returning function	Returns (eligible, reason): drive open, deadline, batch, department, CGPA, backlogs, placement policy, duplicate. Used by the UI **and** enforced by a trigger.
`sp_record_round_result(app, result, score, remarks, evaluator)`	**Procedure**	Locks the application (`SELECT … FOR UPDATE`), inserts the round result, then rejects, advances to the next round, or selects the candidate and inserts the offer.
`sp_accept_offer(offer, student, INOUT withdrawn)`	**Procedure**	Locks the offer, validates owner, status and deadline, accepts it, marks the student placed, auto-declines other pending offers, and withdraws applications the policy no longer allows. Returns the number withdrawn.

7.4 Triggers

Trigger	Timing	Purpose
trg_application_eligibility	BEFORE INSERT on applications	Calls `fn_check_eligibility` and raises an exception with the reason if not eligible. Initialises status and round, and stamps `match_score`.
trg_application_status_guard	BEFORE UPDATE on applications	Final states (selected / rejected / withdrawn) can't change. Maintains updated_at.
trg_offer_requires_selection	BEFORE INSERT on offers	Offers only for selected applications

Trigger	Timing	Purpose
trg_student_resume_changed	BEFORE UPDATE on students	Clears a stale embedding when resume text changes without a new vector
trg_drive_status_guard	BEFORE UPDATE OF status on drives	A completed drive can't be reopened
trg_audit_applications / offers / drives (`05_audit.sql`)	AFTER INSERT/UPDATE/DELETE	Generic `trg_fn_audit()` writes JSONB old/new rows and the acting user, read from the transaction-local setting `app.user_id` set by the API

7.5 Views (`04_views.sql`)

View	Technique
v_drive_summary	LEFT JOIN + GROUP BY + `COUNT(*) FILTER (WHERE …)` funnel per drive
v_dept_placement_stats	LEFT JOIN LATERAL (best accepted offer per student), placement % per department and batch
v_company_offer_stats	Multi-join + GROUP BY + HAVING COUNT ≥ 1
v_student_overview	Scalar subqueries + `string_agg` for skills and counters

7.6 Transactions

Every multi-statement write goes through `withTransaction()` in `src/lib/db.ts`: `BEGIN`, `set_config('app.user_id', …, true)`, the work, then `COMMIT`, or `ROLLBACK` on any error. Examples:

- **Register**: insert into `users` and `students` together. If the SRN is invalid, the user row is rolled back too.
- **Create drive**: drive, departments, skills and rounds are all inserted together, or none are.
- **Add company**: if the company insert succeeds but the recruiter email is a duplicate, the company is rolled back as well.
- **Accept offer / record result**: the procedures use row-level locks (`FOR UPDATE`) so two concurrent requests can't double-accept or double-grade.

8. Vector database / semantic search

Question	Answer
What is embedded?	**Students:** resume text + skill names. **Drives:** title + job description + required skill names.
Model	`sentence-transformers/all-MiniLM-L6-v2` (ONNX, quantised), run locally in Node.js via `@huggingface/transformers`. Output is 384 dimensions with mean pooling and L2 normalisation. No external API is needed.
Storage	`vector(384)` columns `students.resume_embedding` and `drives.jd_embedding` (pgvector), each with an **HNSW** index using `vector_cosine_ops`.
Similarity metric	**Cosine distance** (`<=>`). Similarity = 1 − distance, shown as a match %.
When computed	When the student saves their profile, when a recruiter creates a drive, and in the seed script. The vector is written in the same transaction as the text, and a trigger nulls stale vectors.
Retrieval	`ORDER BY embedding <=> query_vector LIMIT k` (k-NN), combined in the same SQL with relational filters (eligibility function, drive status, department, CGPA).

Where it is used:

1. **Recommended drives** (student dashboard): nearest open drives to the student's resume vector, filtered by `fn_check_eligibility`.
2. **Applicant ranking** (recruiter drive page): applicants ordered by `match_score`, which the insert trigger computes.
3. **Suggested candidates**: eligible students who haven't applied, nearest to the JD vector.
4. **Talent search / semantic drive search**: the user's free-text query is embedded at request time and compared with the stored vectors. It's shown side by side with PostgreSQL full-text search (`tsvector @@ websearch_to_tsquery`, ranked by `ts_rank`) to show the difference. Keyword search needs the exact words, while semantic search finds paraphrases.

9. Important queries

All of these run live on the admin **SQL Lab** page (see `src/lib/showcase.ts`).

Recommended drives for a student (vector + function + join)

```
SELECT d.drive_id, d.title, c.name AS company, d.ctc_lpa,
       ROUND((1 - (d.jd_embedding <=> s.resume_embedding))::numeric, 4) AS score
FROM students s
JOIN drives d    ON d.status = 'open' AND d.jd_embedding IS NOT NULL
JOIN companies c ON c.company_id = d.company_id
WHERE s.student_id = $1
  AND (SELECT eligible FROM fn_check_eligibility(s.student_id, d.drive_id))
ORDER BY d.jd_embedding <=> s.resume_embedding
LIMIT 5;
```

Drives with at least 3 applicants (GROUP BY + HAVING)

```
SELECT c.name, d.title, COUNT(*) AS applicants, ROUND(AVG(a.match_score), 3) AS avg_match
FROM applications a JOIN drives d ON d.drive_id = a.drive_id JOIN companies c ON c.company_id =
d.company_id
GROUP BY c.name, d.title HAVING COUNT(*) >= 3 ORDER BY avg_match DESC;
```

Students above their department's average CGPA (correlated subquery)

```
SELECT u.full_name, s.cgpa FROM students s JOIN users u ON u.user_id = s.student_id
WHERE s.cgpa > (SELECT AVG(s2.cgpa) FROM students s2 WHERE s2.dept_id = s.dept_id);
```

Top 3 applicants per drive (window function)

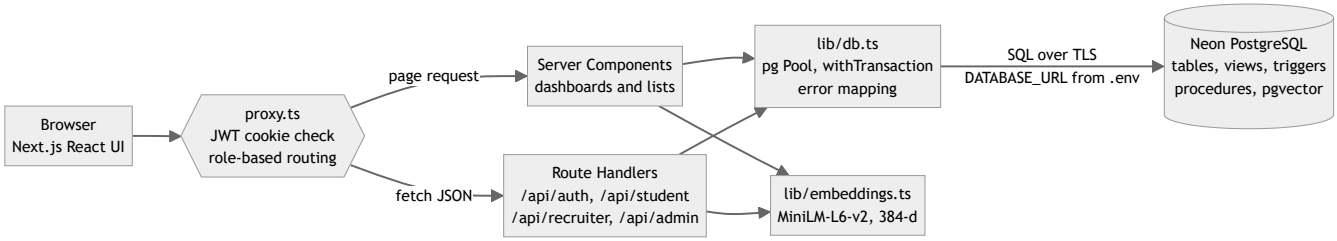
```
WITH ranked AS (
  SELECT a.drive_id, a.student_id, a.match_score,
         RANK() OVER (PARTITION BY a.drive_id ORDER BY a.match_score DESC) AS rnk
  FROM applications a)
SELECT * FROM ranked WHERE rnk <= 3;
```

Unplaced students who never applied (NOT EXISTS), **skills students have that no drive asks for (EXCEPT)**, **round-wise pass percentage (CTE + FILTER)**, **full-text search with ts_rank**, **top-3 nearest students per open drive (LATERAL + `<=>`)** and **EXPLAIN showing the HNSW index** are in the SQL Lab as well.

DML used by the application

- INSERT: register, apply, create drive/rounds, add company, round results, offers (via procedure)
- UPDATE: profile and resume, withdraw, decline/accept offer, drive status, CGPA verification
- DELETE: replace student skills, delete company (CASCADE to drives; RESTRICT when recruiters exist)
- SELECT: every page, using joins, aggregates, subqueries and views

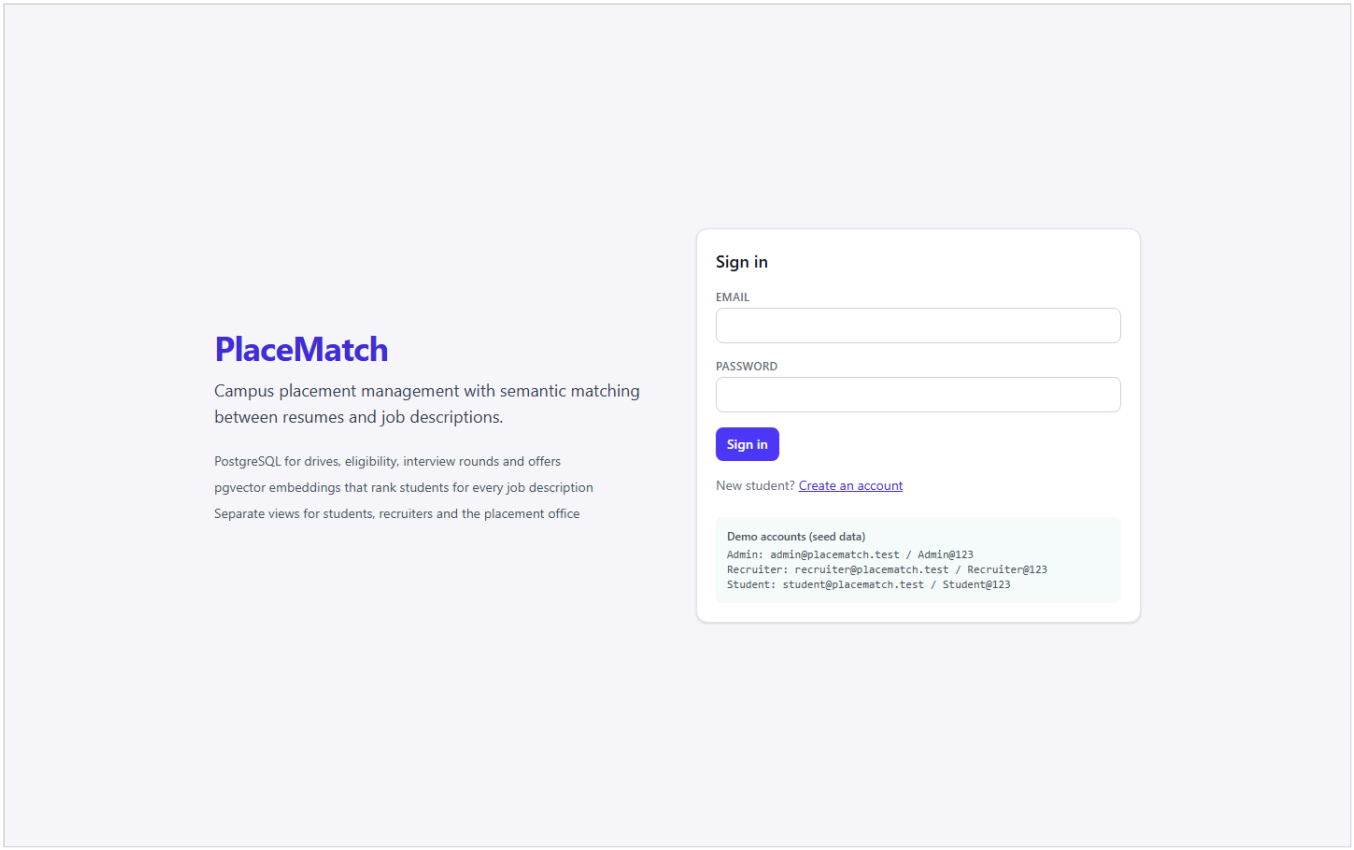
10. System architecture



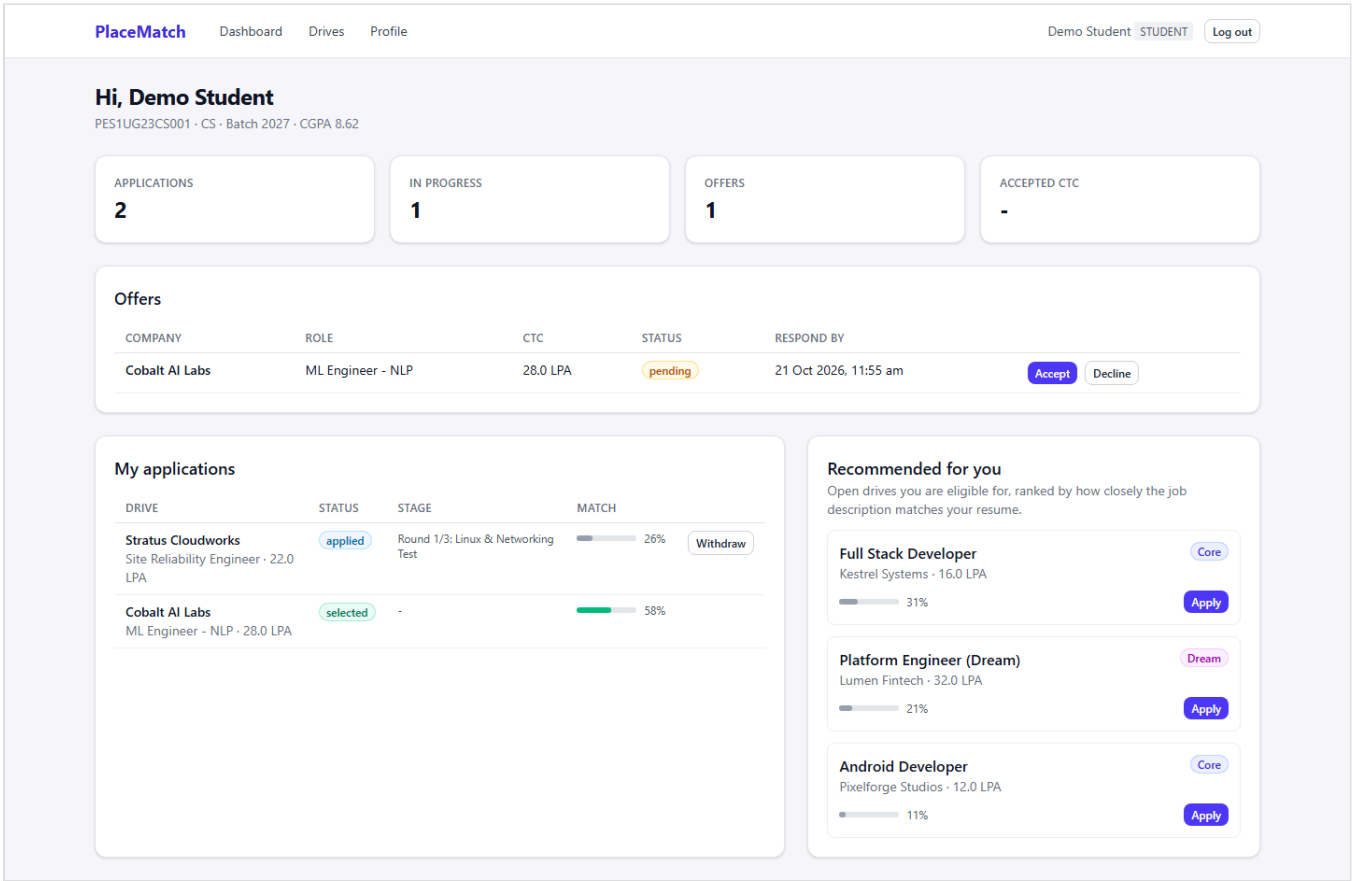
- **Presentation**: Next.js App Router pages per role (`/student`, `/recruiter`, `/admin`).
- **Security**: `src/proxy.ts` checks the signed JWT cookie on every request and redirects or returns 401/403 by role. Every API handler re-checks the role (`requireApi`). Passwords are hashed with bcrypt, and SQL always uses parameterised queries (`$1, $2 …`), so there is no SQL injection.
- **Data access**: one `pg` connection pool configured only from `DATABASE_URL`. Business rules live in the database (constraints, triggers, procedures), so they apply no matter which client writes.

11. Application screenshots

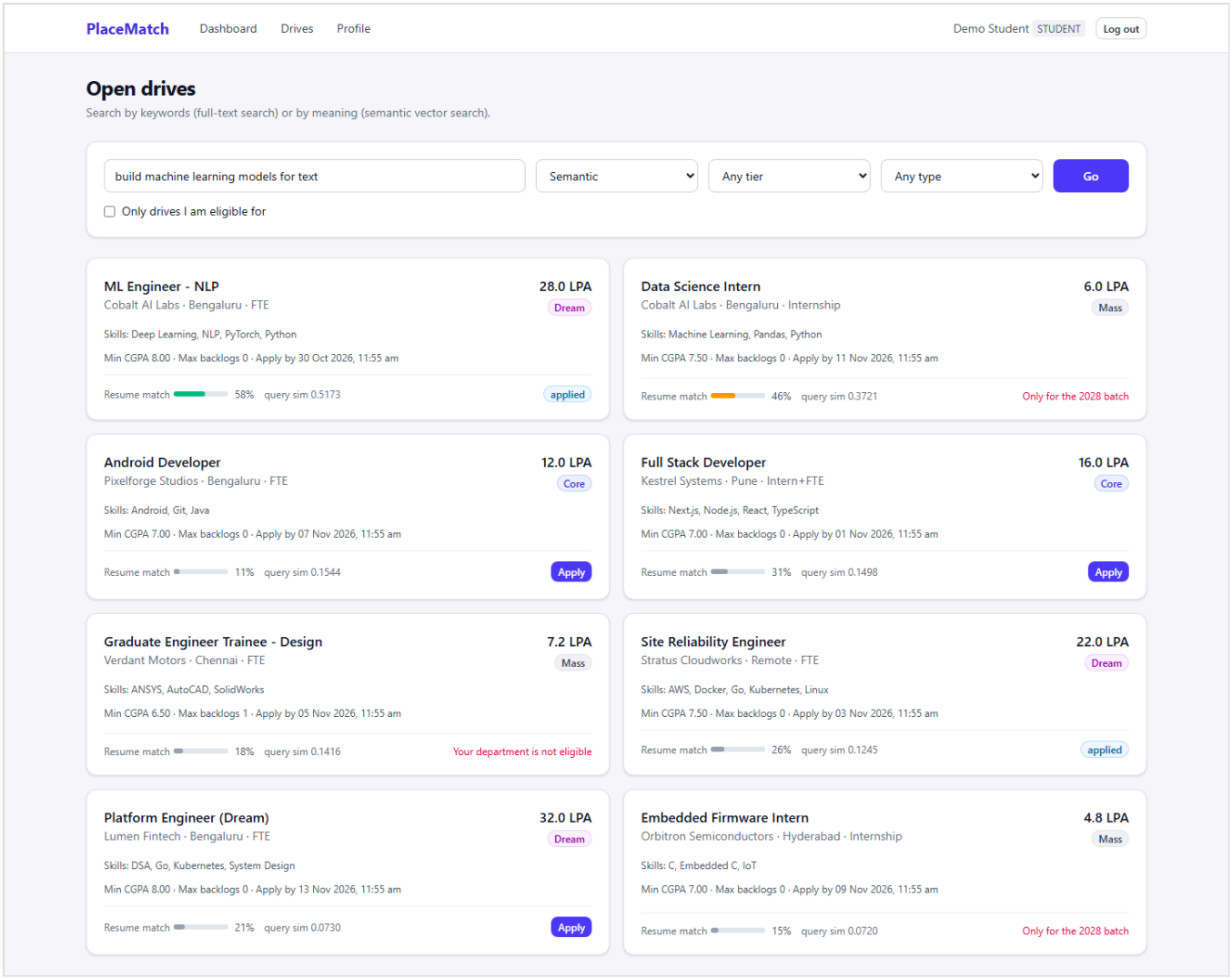
1. Login page with role-based demo accounts



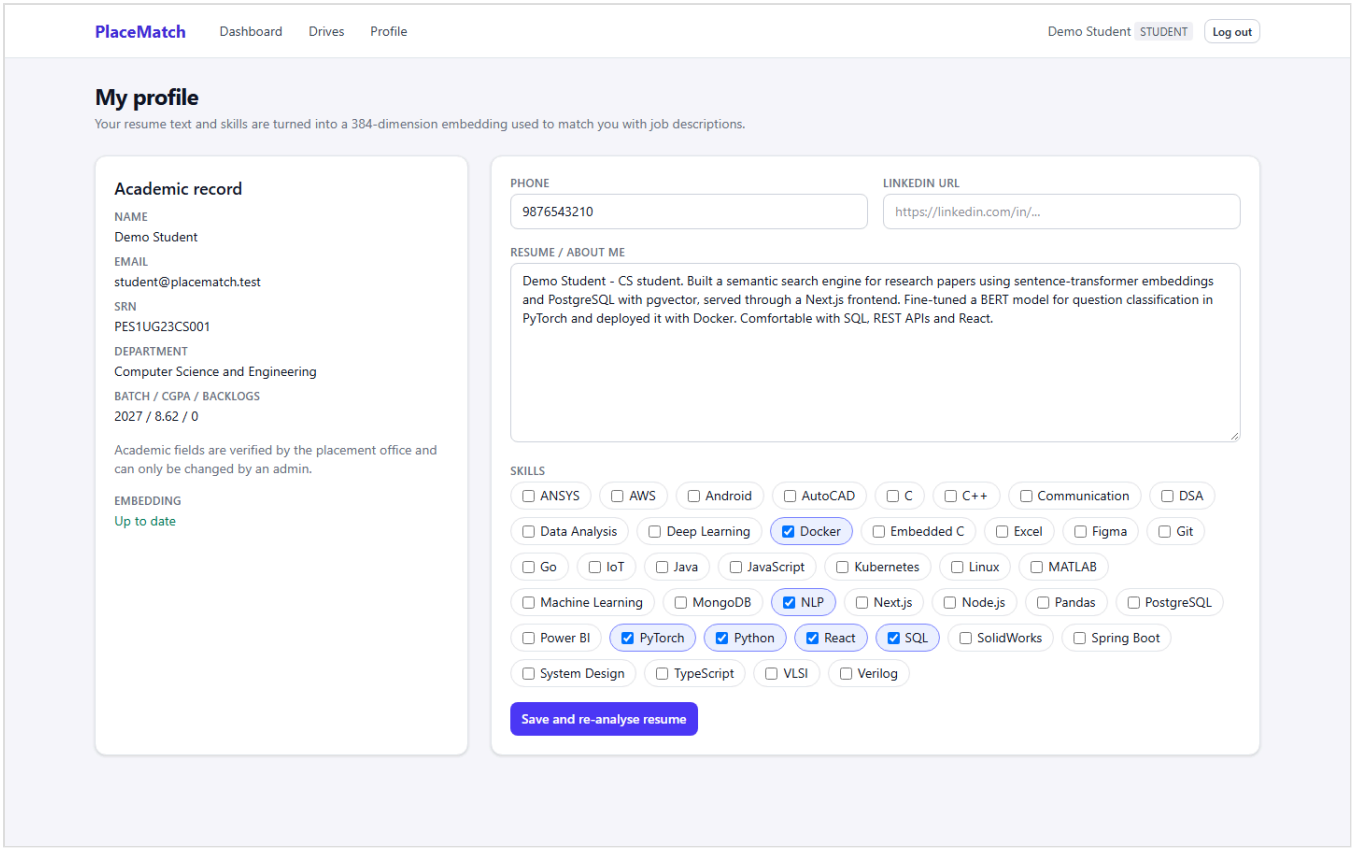
2. Student dashboard: pending offer, round-by-round application status, and recommended drives from vector search



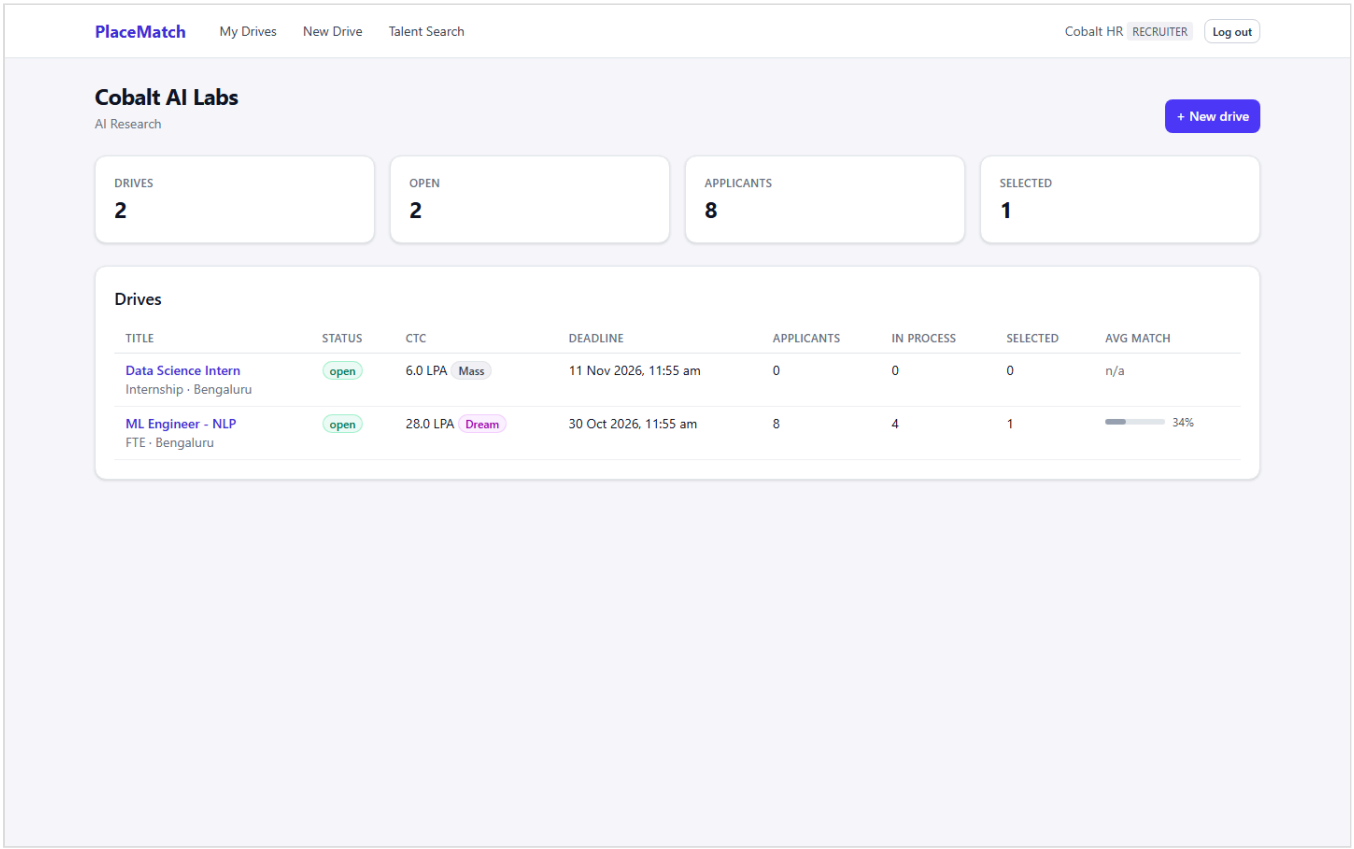
3. Semantic drive search with per-drive eligibility reasons from `fn_check_eligibility`



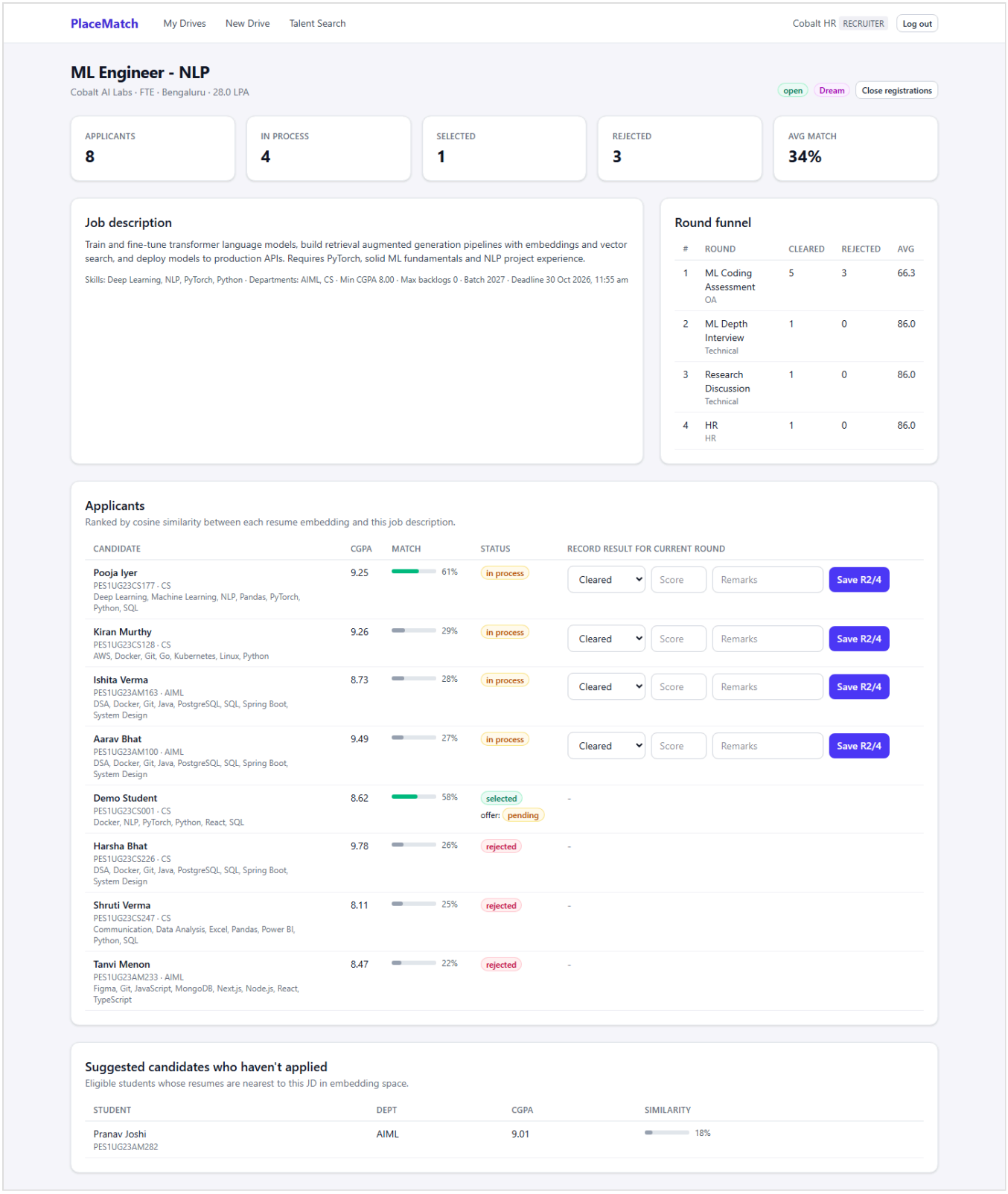
4. Student profile: saving the resume recomputes its embedding



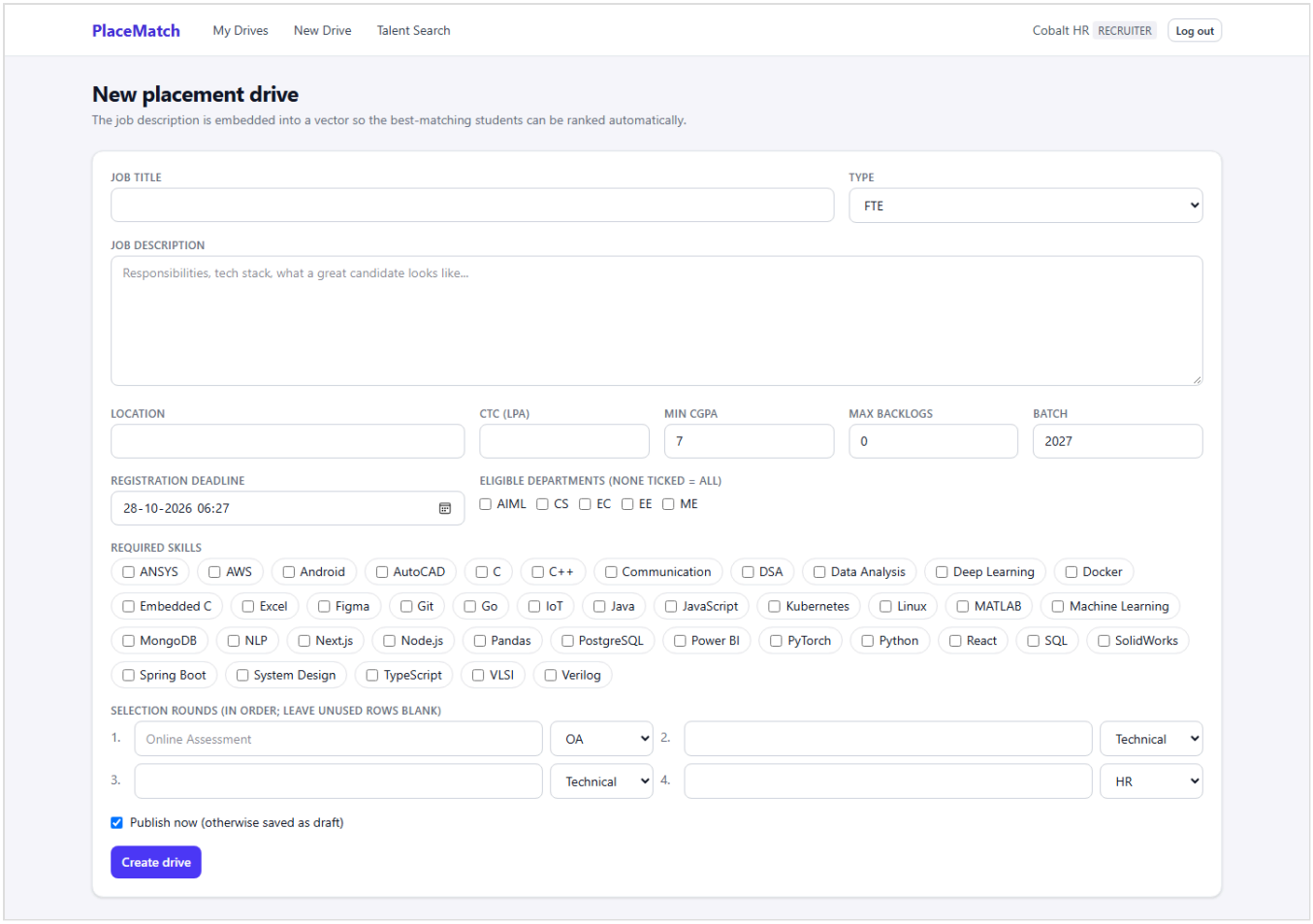
5. Recruiter home: company drives with funnel counts from `v_drive_summary`



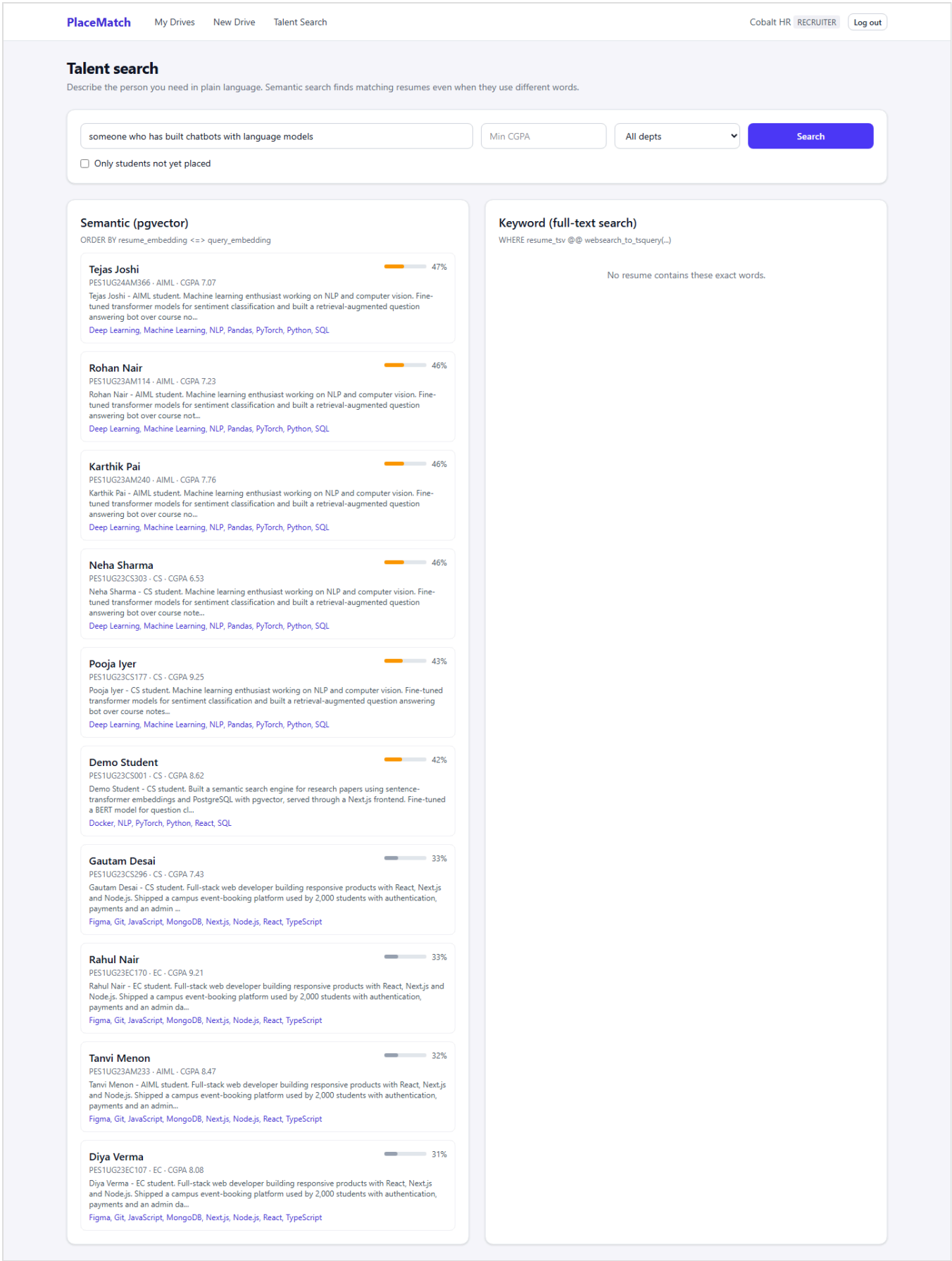
6. Drive detail: round funnel, applicants ranked by match score with round-result entry, and suggested non-applicants



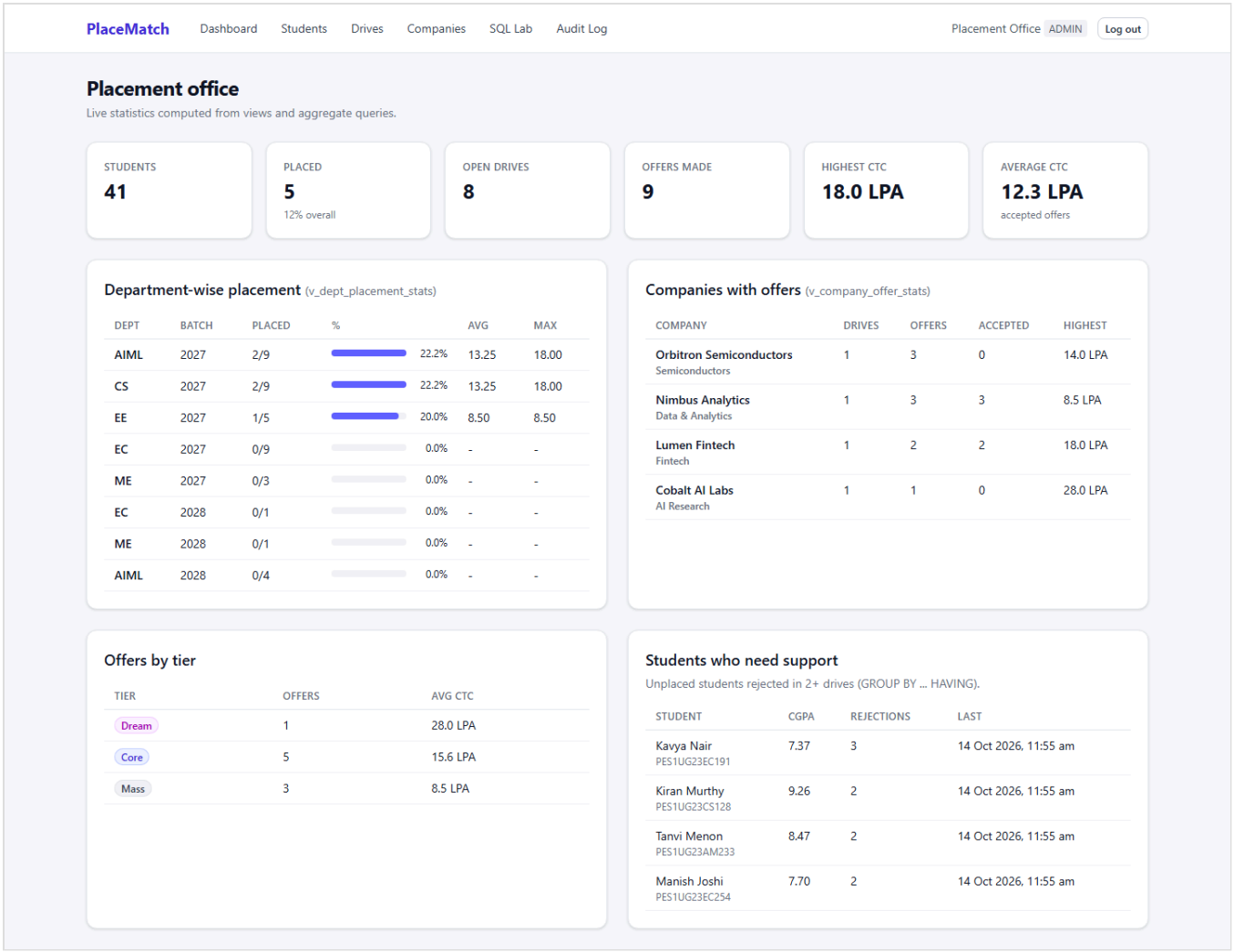
7. Creating a drive with eligibility, departments, skills and rounds (one transaction)



8. Talent search: semantic (pgvector) results versus keyword (full-text) results for the same query



9. Placement office dashboard built from views and GROUP BY / HAVING queries



10. Student list (`v_student_overview`) with CGPA and backlog verification

PlaceMatch Dashboard Students Drives Companies SQL Lab Audit Log Placement Office ADMIN Log out

Students

41 student(s) · data from the v_student_overview view

Name or SRN | All depts | All batches | Placed + unplaced | Filter

STUDENT	DEPT	SKILLS	APPS	STATUS	VERIFY CGPA / BACKLOGS		
Aarav Bhat PES1UG23AM100 · aarav.bhat0@placematch.test	AIML 2027	DSA, Docker, Git, Java, PostgreSQL, SQL, Spring Boot, System Design	3	selected 8.5 LPA	9.49	0	Save
Rohan Nair PES1UG23AM114 · rohan.nair2@placematch.test	AIML 2027	Deep Learning, Machine Learning, NLP, Pandas, PyTorch, Python, SQL	0	unplaced	7.23	0	Save
Varun Kulkarni PES1UG23AM156 · varun.kulkarni8@placematch.test	AIML 2027	Android, Figma, Git, Java, SQL	0	unplaced	8.06	2	Save
Ishita Verma PES1UG23AM163 · ishita.verma9@placematch.test	AIML 2027	DSA, Docker, Git, Java, PostgreSQL, SQL, Spring Boot, System Design	2	selected 18.0 LPA	8.73	0	Save
Tanvi Menon PES1UG23AM233 · tanvi.menon19@placematch.test	AIML 2027	Figma, Git, JavaScript, MongoDB, Next.js, Node.js, React, TypeScript	3	unplaced	8.47	0	Save
Karthik Pai PES1UG23AM240 · karthik.pai20@placematch.test	AIML 2027	Deep Learning, Machine Learning, NLP, Pandas, PyTorch, Python, SQL	0	unplaced	7.76	0	Save
Pranav Joshi PES1UG23AM282 · pranav.joshi26@placematch.test	AIML 2027	Android, Figma, Git, Java, SQL	2	unplaced	9.01	0	Save
Lakshmi Hegde PES1UG23AM289 · lakshmi.hegde27@placematch.test	AIML 2027	DSA, Docker, Git, Java, PostgreSQL, SQL, Spring Boot, System Design	0	unplaced	6.66	0	Save
Suhas Prasad PES1UG23AM310 · suhas.prasad30@placematch.test	AIML 2027	Communication, Data Analysis, Excel, Pandas, Power BI, Python, SQL	0	unplaced	6.31	0	Save
Sanjana Kulkarni PES1UG24AM345 · sanjana.kulkarni35@placematch.test	AIML 2028	Android, Figma, Git, Java, SQL	0	unplaced	6.43	0	Save
Abhishek Shetty PES1UG24AM352 · abhishek.shetty36@placematch.test	AIML 2028	DSA, Docker, Git, Java, PostgreSQL, SQL, Spring Boot, System Design	0	unplaced	7.07	0	Save
Tejas Joshi PES1UG24AM366 · tejas.joshi38@placematch.test	AIML 2028	Deep Learning, Machine Learning, NLP, Pandas, PyTorch, Python, SQL	0	unplaced	7.07	0	Save
Pallavi Sharma PES1UG24AM373 · pallavi.sharma39@placematch.test	AIML 2028	Communication, Data Analysis, Excel, Pandas, Power BI, Python, SQL	0	unplaced	6.92	0	Save
Demo Student PES1UG23CS001 · student@placematch.test	CS 2027	Docker, NLP, PyTorch, Python, React, SQL	2	unplaced	8.62	0	Save
Kiran Murthy PES1UG23CS128 · kiran.murthy4@placematch.test	CS 2027	AWS, Docker, Git, Go, Kubernetes, Linux, Python	4	unplaced	9.26	0	Save
Pooja Iyer PES1UG23CS177 · pooja.iyer11@placematch.test	CS 2027	Deep Learning, Machine Learning, NLP, Pandas, PyTorch, Python, SQL	2	unplaced	9.25	0	Save
Nikhil Joshi PES1UG23CS184 · nikhil.joshi12@placematch.test	CS 2027	Communication, Data Analysis, Excel, Pandas, Power BI, Python, SQL	0	unplaced	6.29	0	Save
Nandini Sharma PES1UG23CS219 · nandini.sharma17@placematch.test	CS 2027	Android, Figma, Git, Java, SQL	0	unplaced	6.29	0	Save
Harsha Bhat PES1UG23CS226 · harsha.bhat18@placematch.test	CS 2027	DSA, Docker, Git, Java, PostgreSQL, SQL, Spring Boot, System Design	2	selected 18.0 LPA	9.78	0	Save
Shruti Verma PES1UG23CS247 · shruti.verma21@placematch.test	CS 2027	Communication, Data Analysis, Excel, Pandas, Power BI, Python, SQL	3	selected 8.5 LPA	8.11	0	Save
Gautam Desai PES1UG23CS296 · gautam.desai28@placematch.test	CS 2027	Figma, Git, JavaScript, MongoDB, Next.js, Node.js, React, TypeScript	1	unplaced	7.43	0	Save
Neha Sharma PES1UG23CS303 · neha.sharma29@placematch.test	CS 2027	Deep Learning, Machine Learning, NLP, Pandas, PyTorch, Python, SQL	1	unplaced	6.53	0	Save
Diya Verma PES1UG23EC107 · diya.verma1@placematch.test	EC 2027	Figma, Git, JavaScript, MongoDB, Next.js, Node.js, React, TypeScript	1	unplaced	8.08	0	Save
Meera Kulkarni PES1UG23EC135 · meera.kulkarni5@placematch.test	EC 2027	C, C++, Embedded C, IoT, Linux, MATLAB	0	unplaced	9.23	2	Save
Rahul Nair PES1UG23EC170 · rahul.nair10@placematch.test	EC 2027	Figma, Git, JavaScript, MongoDB, Next.js, Node.js, React, TypeScript	1	unplaced	9.21	0	Save
Kavya Nair PES1UG23EC191 · kavya.nair13@placematch.test	EC 2027	AWS, Docker, Git, Go, Kubernetes, Linux, Python	3	unplaced	7.37	0	Save
Manish Joshi PES1UG23EC254 · manish.joshi22@placematch.test	EC 2027	AWS, Docker, Git, Go, Kubernetes, Linux, Python	3	unplaced	7.70	0	Save
Bhavana Hegde PES1UG23EC261 · bhavana.hegde23@placematch.test	EC 2027	C, C++, Embedded C, IoT, Linux, MATLAB	1	unplaced	8.15	0	Save
Aishwarya Shetty PES1UG23EC317 · aishwarya.shetty31@placematch.test	EC 2027	AWS, Docker, Git, Go, Kubernetes, Linux, Python	2	unplaced	8.44	0	Save
Vikram Bhat PES1UG23EC324 · vikram.bhat32@placematch.test	EC 2027	C, C++, Embedded C, IoT, Linux, MATLAB	1	unplaced	7.03	0	Save
Deepika Hegde PES1UG23EC331 · deepika.hegde33@placematch.test	EC 2027	C, MATLAB, VLSI, Verilog	1	unplaced	9.77	0	Save
Keerthi Reddy PES1UG24EC359 · keerthi.reddy37@placematch.test	EC 2028	Figma, Git, JavaScript, MongoDB, Next.js, Node.js, React, TypeScript	0	unplaced	6.31	0	Save
Ananya Verma PES1UG23EE121 · ananya.verma3@placematch.test	EE 2027	Communication, Data Analysis, Excel, Pandas, Power BI, Python, SQL	1	selected 8.5 LPA	7.50	0	Save

Arjun Naik PES1UG23EE142 · arjun.naik6@placematch.test	EE 2027	C, MATLAB, VLSI, Verilog	0	unplaced	6.61	0	Save
Siddharth Patil PES1UG23EE198 · siddharth.patil14@placematch.test	EE 2027	C, C++, Embedded C, IoT, Linux, MATLAB	1	unplaced	9.10	0	Save
Riya Sharma PES1UG23EE205 · riya.sharma15@placematch.test	EE 2027	C, MATLAB, VLSI, Verilog	0	unplaced	7.17	2	Save
Akash Naik PES1UG23EE268 · akash.naik24@placematch.test	EE 2027	C, MATLAB, VLSI, Verilog	0	unplaced	8.34	2	Save
Sneha Sharma PES1UG23ME149 · sneha.sharma7@placematch.test	ME 2027	ANSYS, AutoCAD, Excel, MATLAB, SolidWorks	1	unplaced	7.17	0	Save
Aditya Nair PES1UG23ME212 · aditya.nair16@placematch.test	ME 2027	ANSYS, AutoCAD, Excel, MATLAB, SolidWorks	0	unplaced	9.13	0	Save
Divya Patil PES1UG23ME275 · divya.patil25@placematch.test	ME 2027	ANSYS, AutoCAD, Excel, MATLAB, SolidWorks	0	unplaced	6.65	2	Save
Yash Kamath PES1UG24ME338 · yash.kamath34@placematch.test	ME 2028	ANSYS, AutoCAD, Excel, MATLAB, SolidWorks	0	unplaced	7.24	0	Save

11. All drives with status management

PlaceMatch Dashboard Students Drives Companies SQL Lab Audit Log Placement Office ADMIN Log out

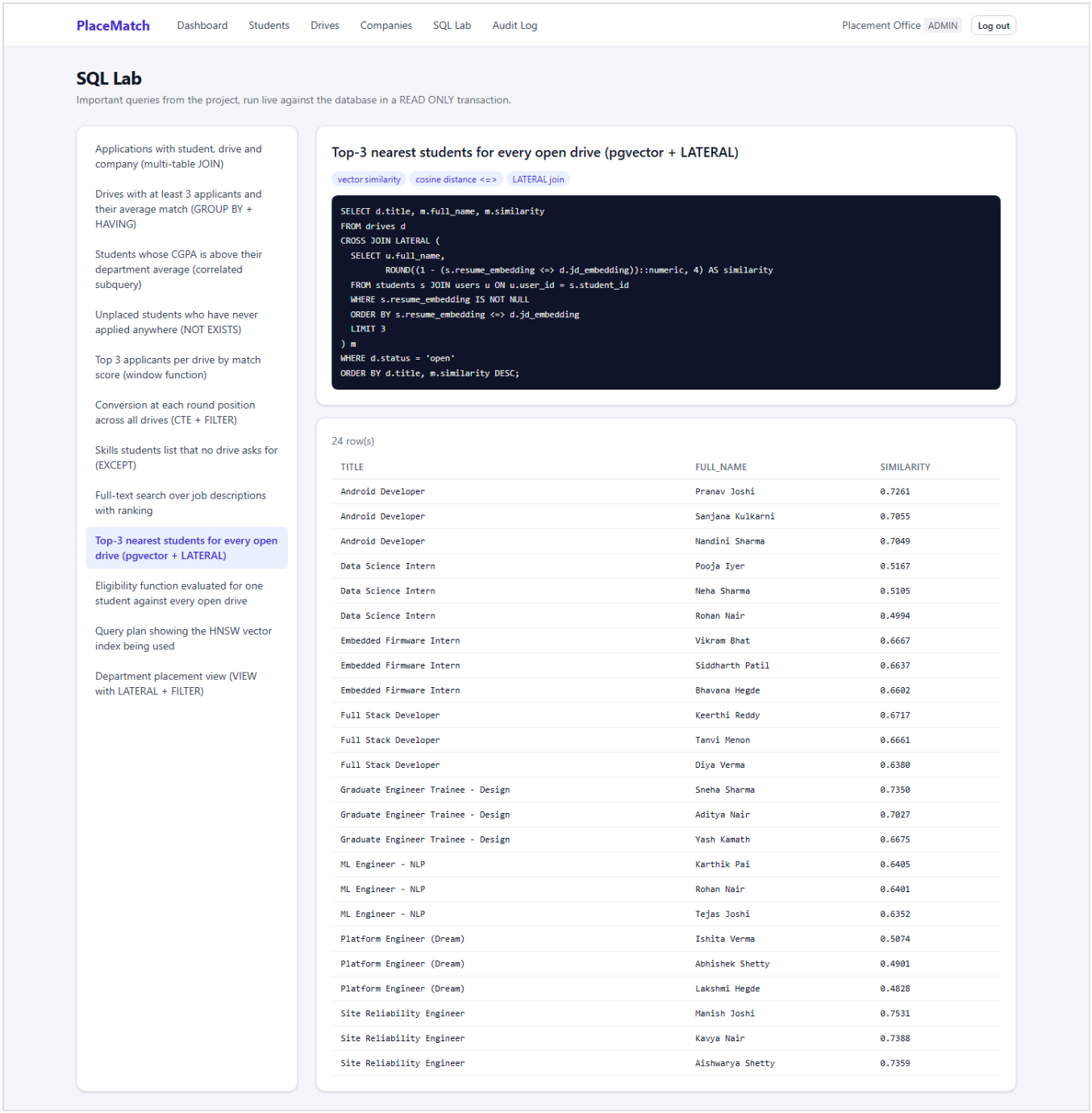
All drives

Drive funnel from the v_drive_summary view.

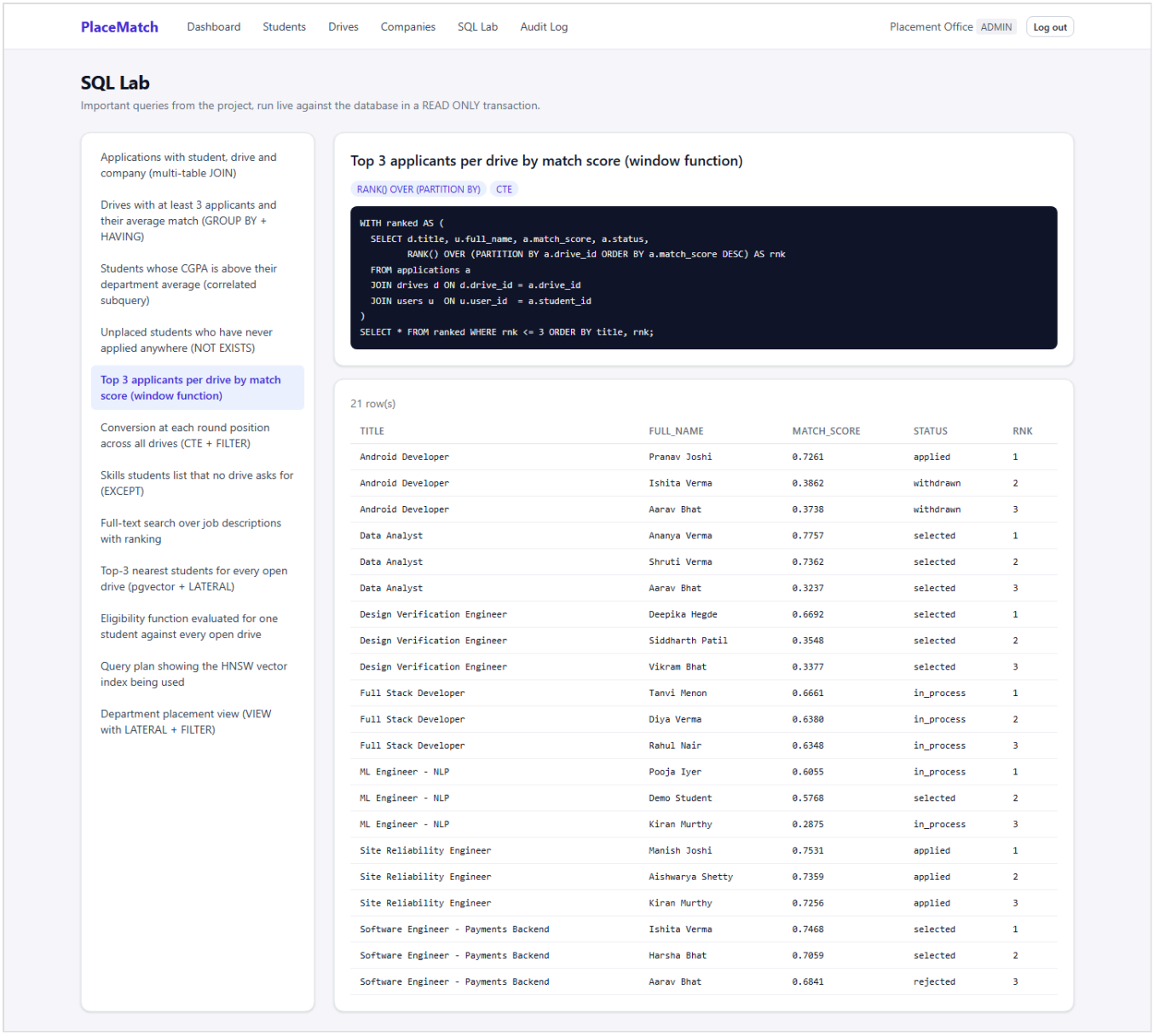
All statuses Filter

DRIVE	CTC	CRITERIA	DEADLINE	APPLICANTS	SELECTED	AVG MATCH	STATUS
Product Analyst Kestrel Systems · FTE	11.0 LPA Core	CGPA ≥ 7.00 Backlogs ≤ 0 Batch 2027	15 Nov 2026, 11:55 am	0	0	n/a	draft Publish
Platform Engineer (Dream) Lumen Fintech · FTE	32.0 LPA Dream	CGPA ≥ 8.00 Backlogs ≤ 0 Batch 2027	13 Nov 2026, 11:55 am	0	0	n/a	open Close
Data Science Intern Cobalt AI Labs · Internship	6.0 LPA Mass	CGPA ≥ 7.50 Backlogs ≤ 0 Batch 2028	11 Nov 2026, 11:55 am	0	0	n/a	open Close
Embedded Firmware Intern Orbitron Semiconductors · Internship	4.8 LPA Mass	CGPA ≥ 7.00 Backlogs ≤ 0 Batch 2028	09 Nov 2026, 11:55 am	0	0	n/a	open Close
Android Developer Pixelforge Studios · FTE	12.0 LPA Core	CGPA ≥ 7.00 Backlogs ≤ 0 Batch 2027	07 Nov 2026, 11:55 am	1	0	73%	open Close
Graduate Engineer Trainee - Design Verdant Motors · FTE	7.2 LPA Mass	CGPA ≥ 6.50 Backlogs ≤ 1 Batch 2027	05 Nov 2026, 11:55 am	0	0	n/a	open Close
Site Reliability Engineer Stratus Cloudworks · FTE	22.0 LPA Dream	CGPA ≥ 7.50 Backlogs ≤ 0 Batch 2027	03 Nov 2026, 11:55 am	4	0	62%	open Close
Full Stack Developer Kestrel Systems · Intern+FTE	16.0 LPA Core	CGPA ≥ 7.00 Backlogs ≤ 0 Batch 2027	01 Nov 2026, 11:55 am	6	0	55%	open Close
ML Engineer - NLP Cobalt AI Labs · FTE	28.0 LPA Dream	CGPA ≥ 8.00 Backlogs ≤ 0 Batch 2027	30 Oct 2026, 11:55 am	8	1	34%	open Close
Data Analyst Nimbus Analytics · FTE	8.5 LPA Mass	CGPA ≥ 6.50 Backlogs ≤ 1 Batch 2027	14 Sept 2026, 11:55 am	8	3	41%	completed Reopen
Design Verification Engineer Orbitron Semiconductors · FTE	14.0 LPA Core	CGPA ≥ 7.00 Backlogs ≤ 0 Batch 2027	14 Sept 2026, 11:55 am	7	3	35%	closed Reopen Complete
Software Engineer - Payments Backend Lumen Fintech · FTE	18.0 LPA Core	CGPA ≥ 7.50 Backlogs ≤ 0 Batch 2027	14 Sept 2026, 11:55 am	7	2	50%	completed Reopen

12. SQL Lab: top-3 nearest students per open drive (LATERAL + `<=>`)



13. SQL Lab: window function ranking



14. Audit log written by triggers, showing field-level changes and the acting user

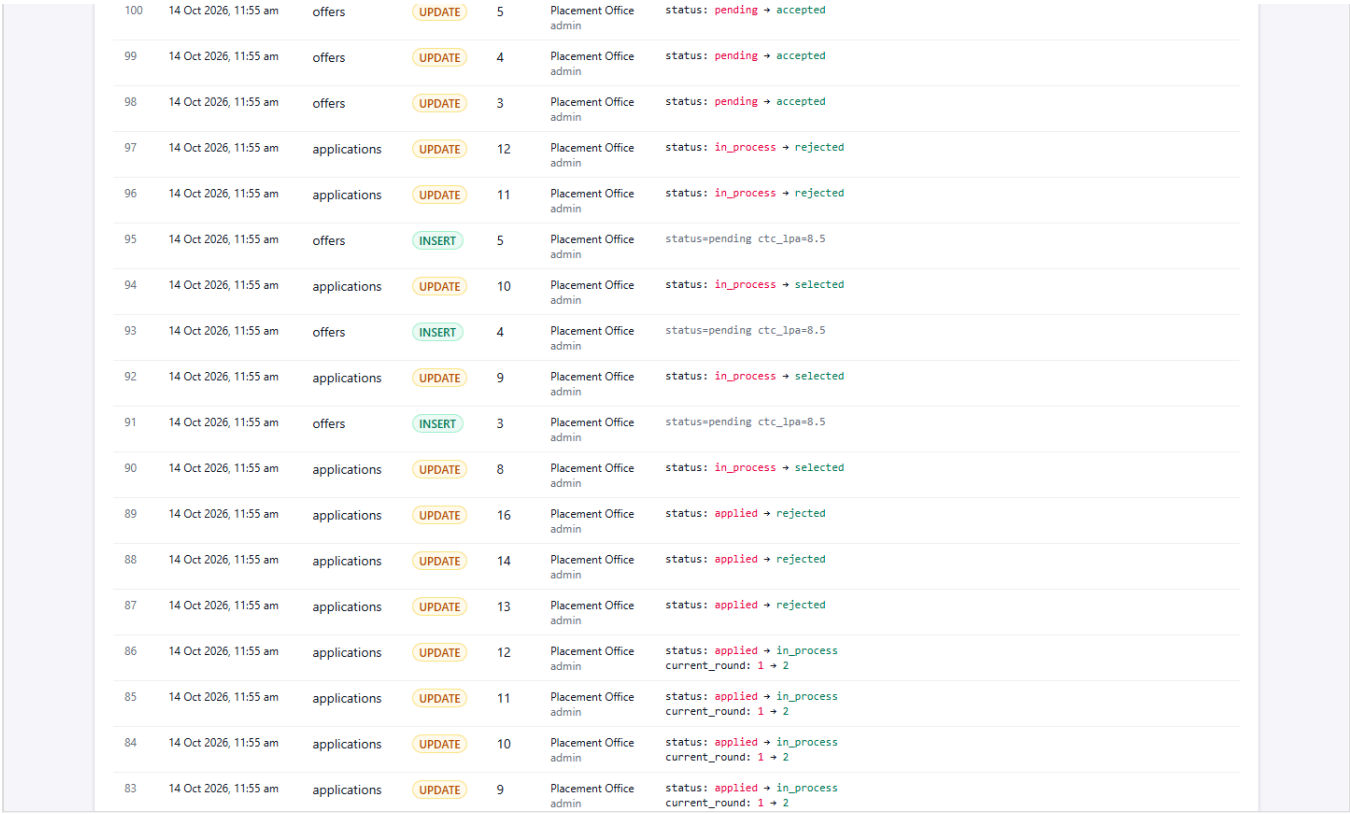
PlaceMatch Dashboard Students Drives Companies SQL Lab Audit Log Placement Office ADMIN Log out

Audit log

Written automatically by AFTER triggers on applications, offers and drives.

All tables Filter

#	WHEN	TABLE	OP	ROW	BY	CHANGE
139	14 Oct 2026, 11:55 am	applications	INSERT	47	Demo Student student	status=applied student_id=10 drive_id=6
138	14 Oct 2026, 11:55 am	offers	INSERT	9	Cobalt HR recruiter	status=pending ctc_lpa=28
137	14 Oct 2026, 11:55 am	applications	UPDATE	25	Cobalt HR recruiter	status: in_process → selected
136	14 Oct 2026, 11:55 am	applications	UPDATE	25	Cobalt HR recruiter	current_round: 3 → 4
135	14 Oct 2026, 11:55 am	applications	UPDATE	25	Cobalt HR recruiter	current_round: 2 → 3
134	14 Oct 2026, 11:55 am	drives	UPDATE	3	Placement Office admin	status: open → closed registration_deadline: 2026-10-28T11:25:12.077+05:00 → 2026-09-14T11:25:12.077+05:00
133	14 Oct 2026, 11:55 am	drives	UPDATE	2	Placement Office admin	status: open → completed registration_deadline: 2026-10-26T11:25:12.077+05:00 → 2026-09-14T11:25:12.077+05:00
132	14 Oct 2026, 11:55 am	drives	UPDATE	1	Placement Office admin	status: open → completed registration_deadline: 2026-10-24T11:25:12.077+05:00 → 2026-09-14T11:25:12.077+05:00
131	14 Oct 2026, 11:55 am	applications	UPDATE	40	Placement Office admin	status: applied → rejected
130	14 Oct 2026, 11:55 am	applications	UPDATE	39	Placement Office admin	status: applied → rejected
129	14 Oct 2026, 11:55 am	applications	UPDATE	35	Placement Office admin	status: applied → in_process current_round: 1 → 2
128	14 Oct 2026, 11:55 am	applications	UPDATE	34	Placement Office admin	status: applied → in_process current_round: 1 → 2
127	14 Oct 2026, 11:55 am	applications	UPDATE	33	Placement Office admin	status: applied → in_process current_round: 1 → 2
126	14 Oct 2026, 11:55 am	applications	UPDATE	32	Placement Office admin	status: applied → in_process current_round: 1 → 2
125	14 Oct 2026, 11:55 am	applications	UPDATE	31	Placement Office admin	status: applied → rejected
124	14 Oct 2026, 11:55 am	applications	UPDATE	30	Placement Office admin	status: applied → rejected
123	14 Oct 2026, 11:55 am	applications	UPDATE	29	Placement Office admin	status: applied → rejected
122	14 Oct 2026, 11:55 am	applications	UPDATE	28	Placement Office admin	status: applied → in_process current_round: 1 → 2
121	14 Oct 2026, 11:55 am	applications	UPDATE	27	Placement Office admin	status: applied → in_process current_round: 1 → 2
120	14 Oct 2026, 11:55 am	applications	UPDATE	26	Placement Office admin	status: applied → in_process current_round: 1 → 2
119	14 Oct 2026, 11:55 am	applications	UPDATE	25	Placement Office admin	status: applied → in_process current_round: 1 → 2
118	14 Oct 2026, 11:55 am	applications	UPDATE	24	Placement Office admin	status: applied → in_process current_round: 1 → 2
117	14 Oct 2026, 11:55 am	applications	UPDATE	21	Placement Office admin	status: in_process → rejected
116	14 Oct 2026, 11:55 am	applications	UPDATE	20	Placement Office admin	status: in_process → rejected
115	14 Oct 2026, 11:55 am	offers	INSERT	8	Placement Office admin	status=pending ctc_lpa=14
114	14 Oct 2026, 11:55 am	applications	UPDATE	19	Placement Office admin	status: in_process → selected
113	14 Oct 2026, 11:55 am	offers	INSERT	7	Placement Office admin	status=pending ctc_lpa=14
112	14 Oct 2026, 11:55 am	applications	UPDATE	18	Placement Office admin	status: in_process → selected
111	14 Oct 2026, 11:55 am	offers	INSERT	6	Placement Office admin	status=pending ctc_lpa=14
110	14 Oct 2026, 11:55 am	applications	UPDATE	17	Placement Office admin	status: in_process → selected
109	14 Oct 2026, 11:55 am	applications	UPDATE	23	Placement Office admin	status: applied → rejected
108	14 Oct 2026, 11:55 am	applications	UPDATE	22	Placement Office admin	status: applied → rejected
107	14 Oct 2026, 11:55 am	applications	UPDATE	21	Placement Office admin	status: applied → in_process current_round: 1 → 2
106	14 Oct 2026, 11:55 am	applications	UPDATE	20	Placement Office admin	status: applied → in_process current_round: 1 → 2
105	14 Oct 2026, 11:55 am	applications	UPDATE	19	Placement Office admin	status: applied → in_process current_round: 1 → 2
104	14 Oct 2026, 11:55 am	applications	UPDATE	18	Placement Office admin	status: applied → in_process current_round: 1 → 2
103	14 Oct 2026, 11:55 am	applications	UPDATE	17	Placement Office admin	status: applied → in_process current_round: 1 → 2
102	14 Oct 2026, 11:55 am	applications	UPDATE	46	Placement Office admin	status: applied → withdrawn
101	14 Oct 2026, 11:55 am	applications	UPDATE	38	Placement Office admin	status: applied → withdrawn



100	14 Oct 2026, 11:55 am	offers	UPDATE	5	Placement Office admin	status: pending → accepted
99	14 Oct 2026, 11:55 am	offers	UPDATE	4	Placement Office admin	status: pending → accepted
98	14 Oct 2026, 11:55 am	offers	UPDATE	3	Placement Office admin	status: pending → accepted
97	14 Oct 2026, 11:55 am	applications	UPDATE	12	Placement Office admin	status: in_process → rejected
96	14 Oct 2026, 11:55 am	applications	UPDATE	11	Placement Office admin	status: in_process → rejected
95	14 Oct 2026, 11:55 am	offers	INSERT	5	Placement Office admin	status=pending ctc_lpa=8.5
94	14 Oct 2026, 11:55 am	applications	UPDATE	10	Placement Office admin	status: in_process → selected
93	14 Oct 2026, 11:55 am	offers	INSERT	4	Placement Office admin	status=pending ctc_lpa=8.5
92	14 Oct 2026, 11:55 am	applications	UPDATE	9	Placement Office admin	status: in_process → selected
91	14 Oct 2026, 11:55 am	offers	INSERT	3	Placement Office admin	status=pending ctc_lpa=8.5
90	14 Oct 2026, 11:55 am	applications	UPDATE	8	Placement Office admin	status: in_process → selected
89	14 Oct 2026, 11:55 am	applications	UPDATE	16	Placement Office admin	status: applied → rejected
88	14 Oct 2026, 11:55 am	applications	UPDATE	14	Placement Office admin	status: applied → rejected
87	14 Oct 2026, 11:55 am	applications	UPDATE	13	Placement Office admin	status: applied → rejected
86	14 Oct 2026, 11:55 am	applications	UPDATE	12	Placement Office admin	status: applied → in_process current_round: 1 → 2
85	14 Oct 2026, 11:55 am	applications	UPDATE	11	Placement Office admin	status: applied → in_process current_round: 1 → 2
84	14 Oct 2026, 11:55 am	applications	UPDATE	10	Placement Office admin	status: applied → in_process current_round: 1 → 2
83	14 Oct 2026, 11:55 am	applications	UPDATE	9	Placement Office admin	status: applied → in_process current_round: 1 → 2

12. Testing

`npm run test:db` runs automated database test cases covering constraints, triggers, procedures, transactions, views, full-text and vector search. Each test runs inside `BEGIN … ROLLBACK`, so it leaves no trace. The results are in `test-results.md`.

Result: 26 / 26 passed.

#	Area	Test case	Result	Observed
1	Constraint	CHECK: malformed SRN is rejected	PASS	new row for relation "students" violates check constraint "chk_srn"
2	Constraint	CHECK: CGPA above 10 is rejected	PASS	new row for relation "students" violates check constraint "chk_cgpa"
3	Constraint	CHECK: invalid email format is rejected	PASS	new row for relation "users" violates check constraint "chk_email_format"
4	Constraint	UNIQUE: duplicate email is rejected	PASS	duplicate key value violates unique constraint "users_email_key"
5	Constraint	FOREIGN KEY: student in a non-existent department is rejected	PASS	insert or update on table "students" violates foreign key constraint "fk_students_dept"
6	Constraint	ENUM: unknown application status is rejected	PASS	invalid input value for enum app_status: "hired"
7	Trigger	TRIGGER: eligible student can apply; match score is stamped automatically	PASS	match_score = 1.0000
8	Trigger	TRIGGER: CGPA below drive minimum is blocked	PASS	Not eligible: Minimum CGPA is 8.00 (yours: 6.10)
9	Trigger	TRIGGER: department not in the drive list is blocked	PASS	Not eligible: Your department is not eligible

#	Area	Test case	Result	Observed
10	Trigger	TRIGGER: applying to a draft drive is blocked	PASS	Not eligible: Drive is not open for registration
11	Trigger	TRIGGER: duplicate application is blocked	PASS	Not eligible: You have already applied to this drive
12	Procedure	PROCEDURE: clearing round 1 of 2 advances the candidate	PASS	status=in_process, current_round=2
13	Procedure	PROCEDURE: clearing the last round selects the candidate and creates an offer	PASS	offer 14.00 LPA, pending
14	Procedure	PROCEDURE: rejection ends the application; grading it again fails	PASS	Application is already rejected
15	Trigger	TRIGGER: an offer cannot be inserted for a non-selected application	PASS	Offers can only be made to selected candidates
16	Transaction	TRANSACTION: accepting an offer places the student, declines other offers, withdraws non-dream applications	PASS	withdrawn=1; offer2=declined; dream app kept
17	Trigger	POLICY: a placed student cannot apply to a non-Dream drive	PASS	Not eligible: Already placed at 12.00 LPA: only Dream drives paying >= 18.00 LPA allowed
18	Transaction	TRANSACTION: failure inside a transaction rolls back earlier statements	PASS	company insert undone after duplicate recruiter email
19	Trigger	TRIGGER: final application states cannot change	PASS	Cannot change an application that is already withdrawn
20	Trigger	TRIGGER: a completed drive cannot be reopened	PASS	A completed drive cannot be reopened
21	Trigger	TRIGGER: editing resume text clears the stale embedding	PASS	resume_embedding set to NULL
22	Trigger	AUDIT: updates are logged with the acting user (app.user_id)	PASS	logged applied -> withdrawn by user 84
23	Vector	VECTOR: nearest-neighbour search ranks the most similar resume first	PASS	exact match ranked #1 with similarity 1.0
24	Vector	VECTOR: real resume embeddings rank a relevant student above an unrelated one	PASS	5/5 nearest resumes to the NLP job are ML profiles
25	Search	FULL-TEXT: keyword search finds matching job descriptions	PASS	Site Reliability Engineer, Platform Engineer (Dream)
26	View	VIEW: v_drive_summary counts match the base tables	PASS	40 = 40

The API layer was also tested end to end through the running app. Students were blocked from recruiter APIs with 403. Recruiters couldn't grade another company's applications. Duplicate and ineligible applications returned the trigger's message to the UI. CHECK violations (for example a phone number or CTC) came back as readable errors. The full apply → grade → offer → accept flow updated every table consistently.

13. Individual contributions

Area	Swaroop Venkateshwar	Shreesh S A
Database	ER design and schema (`01_schema.sql`), indexes, eligibility function, `sp_record_round_result` / `sp_accept_offer`, triggers	Reporting views (`04_views.sql`), audit trigger and audit log, analytical queries for the SQL Lab
Vector / search	Embedding pipeline (MiniLM, pgvector, HNSW), recommendations, applicant ranking	Full-text search (tsvector + GIN), semantic vs keyword talent search
Application	Student module (dashboard, drive search, profile, offers), DB access layer and transactions	Authentication and role-based access, recruiter module, admin module (dashboard, students, companies, drives, SQL Lab)
Documentation / testing	Database test suite, seed data	Project report, README, screenshots

14. Conclusion

PlaceMatch shows how a relational database can be the core of a real application rather than just storage. Placement rules (eligibility, round progression, offer acceptance and the Dream policy) are encoded in constraints, triggers and stored procedures, so they hold for every client and every concurrent request. Views and aggregate queries give the placement office live statistics without duplicated data. pgvector adds semantic understanding of resumes and job descriptions in the same database, so similarity search combines naturally with relational filters in a single SQL statement, inside the same transactions.

Possible extensions: a hybrid score that blends `ts_rank` with vector similarity, scheduled reminders for offer deadlines, resume PDF upload with text extraction, and row-level security policies per role inside PostgreSQL.